Distribución de Microservicios

Microservicio	Lenguaje	BD	ORM
API Gateway	Java (Spring Boot)	—	—
Identity Service	Java (Spring Boot)	MySQL	Hibernate/JPA
Goal & Task Service	NestJS	PostgreSQL	Prisma
Gamification Service	Python (FastAPI)	MongoDB	Motor (async MongoDB driver) + Pydantic
Challenge Service	NestJS	PostgreSQL	Prisma

CONTRATO COMPLETO DE APIs

MICROSERVICIO 1: IDENTITY SERVICE (Java Spring Boot + MySQL)

Responsabilidades: Registro, autenticación, gestión de usuarios, perfiles, roles y auditoría.

Entidades en BD:

```sql
-- users
CREATE TABLE users (
    id VARCHAR(36) PRIMARY KEY, -- UUID
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    hash_password VARCHAR(255) NOT NULL,
    rol ENUM('USER', 'ADMIN') DEFAULT 'USER',
    avatar_url VARCHAR(500) NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- audit_log
CREATE TABLE audit_log (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id VARCHAR(36) NOT NULL,
    action VARCHAR(100) NOT NULL, -- LOGIN, LOGOUT, DELETE_TASK,
CHALLENGE_CREATED, etc.
    description TEXT,
    ip_address VARCHAR(45),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Endpoints:

1. POST /auth/register

Registrar un nuevo usuario.

```
Request Body:
{
    "name": "string",          // requerido, min 2, max 100
    "email": "string",         // requerido, formato email válido
    "password": "string"       // requerido, min 8 caracteres
}

Response 201:
{
    "id": "uuid-string",
    "name": "string",
    "email": "string",
    "rol": "USER",
    "created_at": "2025-01-15T10:30:00Z"
}

Response 400:
{
    "error": "VALIDATION_ERROR",
    "message": "El email ya está registrado"
}
```

2. POST /auth/login

Autenticar usuario y retornar JWT.

```
Request Body:
{
    "email": "string",
    "password": "string"
}

Response 200:
{
    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "token_type": "Bearer",
    "expires_in": 3600,
    "user": {
        "id": "uuid-string",
        "name": "string",
        "email": "string",
        "rol": "USER"
    }
}
```

```
Response 401:
{
    "error": "INVALID_CREDENTIALS",
    "message": "Email o contraseña incorrectos"
}
```

---

3. POST /auth/logout

Invalidar sesión (registrar en auditoría).

```
Headers: Authorization: Bearer <token>

Response 200:
{
    "message": "Sesión cerrada exitosamente"
}
```

---

4. GET /users/profile

Obtener perfil del usuario autenticado.

```
Headers: Authorization: Bearer <token>

Response 200:
{
    "id": "uuid-string",
    "name": "string",
    "email": "string",
    "rol": "USER",
    "avatar_url": "string | null",
    "created_at": "2025-01-15T10:30:00Z"
}
```

---

5. PATCH /users/profile

Editar perfil del usuario autenticado.

```
Headers: Authorization: Bearer <token>

Request Body (todos opcionales):
{
    "name": "string",
    "avatar_url": "string"
```

```
}

Response 200:
{
    "id": "uuid-string",
    "name": "string",
    "email": "string",
    "avatar_url": "string",
    "updated_at": "2025-01-15T12:00:00Z"
}
```

---

6. GET /users/{userId}

Obtener información pública de un usuario (usado internamente por Challenge Service para verificar que el rival existe).

```
Headers: Authorization: Bearer <token>

Path Params:
- userId: string (UUID)

Response 200:
{
    "id": "uuid-string",
    "name": "string",
    "avatar_url": "string | null",
    "rol": "USER"
}

Response 404:
{
    "error": "USER_NOT_FOUND",
    "message": "El usuario no existe"
}
```

---

7. GET /users

Listar usuarios (solo ADMIN) o buscar usuarios por nombre/email (para desafíos).

```
Headers: Authorization: Bearer <token>

Query Params:
- search: string (opcional, busca por nombre o email)
- page: int (default 1)
- limit: int (default 20)

Response 200:
```

```
{
    "data": [
        {
            "id": "uuid-string",
            "name": "string",
            "email": "string",
            "rol": "USER",
            "created_at": "2025-01-15T10:30:00Z"
        }
    ],
    "pagination": {
        "page": 1,
        "limit": 20,
        "total": 150
    }
}
```

---

8. POST /auth/validate-token

Endpoint interno para que otros microservicios validen el JWT sin conocer el secreto.

```
Request Body:
{
    "token": "eyJhbGciOiJIUzI1NiIs..."
}

Response 200:
{
    "valid": true,
    "user_id": "uuid-string",
    "rol": "USER",
    "email": "string"
}

Response 401:
{
    "valid": false,
    "message": "Token inválido o expirado"
}
```

---

9. GET /audit/logs

Obtener logs de auditoría (solo ADMIN).

```
Headers: Authorization: Bearer <token>

Query Params:
```

```
- user_id: string (opcional)
- action: string (opcional)
- from: date (opcional)
- to: date (opcional)
- page: int (default 1)
- limit: int (default 50)

Response 200:
{
    "data": [
        {
            "id": 1,
            "user_id": "uuid-string",
            "action": "LOGIN",
            "description": "Inicio de sesión exitoso",
            "ip_address": "192.168.1.1",
            "created_at": "2025-01-15T10:30:00Z"
        }
    ],
    "pagination": {
        "page": 1,
        "limit": 50,
        "total": 500
    }
}
```

---

10. POST /audit/logs

Endpoint interno para que otros microservicios registren acciones de auditoría.

```
Request Body:
{
    "user_id": "uuid-string",
    "action": "DELETE_TASK",          // LOGIN, LOGOUT, DELETE_TASK,
DELETE_GOAL, CHALLENGE_CREATED, CHALLENGE_ACCEPTED, etc.
    "description": "El usuario eliminó la tarea con id 45",
    "ip_address": "192.168.1.1"       // opcional
}

Response 201:
{
    "id": 1,
    "message": "Log registrado exitosamente"
}
```

MICROSERVICIO 2: GOAL & TASK SERVICE (NestJS + PostgreSQL + Prisma)

Responsabilidades: CRUD completo de metas y tareas, marcar como completada, regeneración automática de tareas recurrentes, cálculo de progreso de metas.

Schema Prisma:

```
enum StateGoal {
    pending
    completed
    expired
}

enum DifficultyLevel {
    easy
    middle
    high
}

model Goal {
    id          Int       @id @default(autoincrement())
    userId      String    // UUID del Identity Service
    name        String
    description String?
    endDate     DateTime?
    state       StateGoal @default(pending)
    createdAt   DateTime  @default(now())
    maxDaysLater Int?     // días máximos de tolerancia
    tasks       Task[]
}

model Task {
    id              Int             @id @default(autoincrement())
    goalId          Int
    goal            Goal            @relation(fields: [goalId], references:
[id])
    name            String
    state           StateGoal       @default(pending)
    difficultyLevel DifficultyLevel
    limitDate       DateTime?
    frequency       Int             // en días: 1=diaria, 7=semanal,
15=quincenal, 0=no recurrente
    createdAt       DateTime        @default(now())
    completions     TaskCompletion[]
}

model TaskCompletion {
    id          Int      @id @default(autoincrement())
    taskId      Int
    task        Task     @relation(fields: [taskId], references: [id])
    completedAt DateTime @default(now())
    xpAwarded   Int
}
```

Endpoints:

METAS (Goals)

1. POST /goals

Crear una meta.

```
Headers: Authorization: Bearer <token>

Request Body:
{
    "name": "string",              // requerido, min 1, max 200
    "description": "string",       // opcional
    "endDate": "2025-03-01",       // opcional, formato ISO
    "maxDaysLater": 3              // opcional, días de tolerancia
}

Response 201:
{
    "id": 1,
    "userId": "uuid-string",
    "name": "Aprender inglés",
    "description": "Estudiar inglés todos los días",
    "endDate": "2025-03-01T00:00:00Z",
    "state": "pending",
    "maxDaysLater": 3,
    "createdAt": "2025-01-15T10:30:00Z",
    "tasks": []
}
```

---

2. GET /goals

Listar todas las metas del usuario autenticado.

```
Headers: Authorization: Bearer <token>

Query Params:
- state: string (opcional: "pending", "completed", "expired")
- page: int (default 1)
- limit: int (default 10)

Response 200:
{
    "data": [
        {
```

```
            "id": 1,
            "userId": "uuid-string",
            "name": "Aprender inglés",
            "description": "Estudiar inglés todos los días",
            "endDate": "2025-03-01T00:00:00Z",
            "state": "pending",
            "maxDaysLater": 3,
            "createdAt": "2025-01-15T10:30:00Z",
            "progress": 0.45,        // porcentaje calculado
            "tasks": [
                {
                    "id": 1,
                    "name": "Estudiar vocabulario",
                    "state": "completed",
                    "difficultyLevel": "middle",
                    "frequency": 1
                }
            ]
        }
    ],
    "pagination": {
        "page": 1,
        "limit": 10,
        "total": 5
    }
}
```

---

3. GET /goals/:id

Obtener detalle de una meta.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "id": 1,
    "userId": "uuid-string",
    "name": "Aprender inglés",
    "description": "Estudiar inglés todos los días",
    "endDate": "2025-03-01T00:00:00Z",
    "state": "pending",
    "maxDaysLater": 3,
    "createdAt": "2025-01-15T10:30:00Z",
    "progress": 0.45,
    "tasks": [
        {
            "id": 1,
```

```
            "name": "Estudiar vocabulario",
            "state": "pending",
            "difficultyLevel": "middle",
            "limitDate": "2025-01-20T00:00:00Z",
            "frequency": 1,
            "createdAt": "2025-01-15T10:30:00Z"
        }
    ]
}

Response 404:
{
    "error": "GOAL_NOT_FOUND",
    "message": "La meta no existe"
}
```

---

4. PATCH /goals/:id

Editar una meta.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Request Body (todos opcionales):
{
    "name": "string",
    "description": "string",
    "endDate": "2025-04-01",
    "maxDaysLater": 5
}

Response 200:
{
    "id": 1,
    "name": "Aprender inglés avanzado",
    "description": "...",
    "endDate": "2025-04-01T00:00:00Z",
    "state": "pending",
    "maxDaysLater": 5,
    "createdAt": "2025-01-15T10:30:00Z"
}
```

---

5. DELETE /goals/:id

Eliminar una meta y todas sus tareas asociadas.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "message": "Meta eliminada exitosamente"
}

Response 404:
{
    "error": "GOAL_NOT_FOUND",
    "message": "La meta no existe"
}
```

---

6. PATCH /goals/:id/reopen

Reabrir una meta completada o expirada.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "id": 1,
    "state": "pending",
    "message": "Meta reabierta exitosamente"
}
```

TAREAS (Tasks)

7. POST /tasks

Crear una tarea asociada a una meta.

```
Headers: Authorization: Bearer <token>

Request Body:
{
    "goalId": 1,                        // requerido
    "name": "string",                   // requerido
    "difficultyLevel": "easy",          // requerido: "easy" | "middle" | "high"
```

```
    "limitDate": "2025-02-01",         // opcional
    "frequency": 1                     // requerido: 0=única, 1=diaria,
7=semanal, 15=quincenal
}

Response 201:
{
    "id": 1,
    "goalId": 1,
    "name": "Estudiar vocabulario",
    "state": "pending",
    "difficultyLevel": "easy",
    "limitDate": "2025-02-01T00:00:00Z",
    "frequency": 1,
    "createdAt": "2025-01-15T10:30:00Z"
}

Response 404:
{
    "error": "GOAL_NOT_FOUND",
    "message": "La meta asociada no existe"
}
```

---

8. GET /tasks

Listar tareas del usuario autenticado.

```
Headers: Authorization: Bearer <token>

Query Params:
- goalId: int (opcional, filtrar por meta)
- state: string (opcional: "pending", "completed", "expired")
- frequency: int (opcional)
- date: string (opcional, "2025-01-15", filtra tareas activas para ese día)
- page: int (default 1)
- limit: int (default 20)

Response 200:
{
    "data": [
        {
            "id": 1,
            "goalId": 1,
            "goalName": "Aprender inglés",
            "name": "Estudiar vocabulario",
            "state": "pending",
            "difficultyLevel": "easy",
            "limitDate": "2025-02-01T00:00:00Z",
            "frequency": 1,
            "createdAt": "2025-01-15T10:30:00Z",
```

```
            "lastCompletedAt": "2025-01-14T18:00:00Z"  // última vez completada
o null
        }
    ],
    "pagination": {
        "page": 1,
        "limit": 20,
        "total": 15
    }
}
```

---

9. GET /tasks/:id

Obtener detalle de una tarea.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "id": 1,
    "goalId": 1,
    "goalName": "Aprender inglés",
    "name": "Estudiar vocabulario",
    "state": "pending",
    "difficultyLevel": "easy",
    "limitDate": "2025-02-01T00:00:00Z",
    "frequency": 1,
    "createdAt": "2025-01-15T10:30:00Z",
    "completions": [
        {
            "id": 1,
            "completedAt": "2025-01-14T18:00:00Z",
            "xpAwarded": 25
        }
    ]
}
```

---

10. PATCH /tasks/:id

Editar una tarea.

```
Headers: Authorization: Bearer <token>

Path Params:
```

```
- id: int

Request Body (todos opcionales):
{
    "name": "string",
    "difficultyLevel": "high",
    "limitDate": "2025-03-01",
    "frequency": 7
}

Response 200:
{
    "id": 1,
    "goalId": 1,
    "name": "Estudiar vocabulario avanzado",
    "state": "pending",
    "difficultyLevel": "high",
    "limitDate": "2025-03-01T00:00:00Z",
    "frequency": 7,
    "createdAt": "2025-01-15T10:30:00Z"
}
```

---

11. DELETE /tasks/:id

Eliminar una tarea.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "message": "Tarea eliminada exitosamente"
}
```

---

12. PATCH /tasks/:id/complete

Marcar tarea como completada. Este es el endpoint más crítico del sistema.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Request Body: (vacío)
```

```
Lógica interna:
1. Verificar que la tarea existe y pertenece al usuario
2. Verificar elegibilidad (no completada ya hoy si es recurrente)
3. Actualizar state → completed
4. Llamar al Gamification Service: POST /gamification/task-completed
5. Si es recurrente (frequency > 0), generar siguiente instancia
6. Retornar resultado combinado

Response 200:
{
    "task": {
        "id": 1,
        "name": "Estudiar vocabulario",
        "state": "completed",
        "completedAt": "2025-01-15T18:00:00Z"
    },
    "gamification": {
        "xpAwarded": 35,
        "totalXp": 1250,
        "currentLevel": 5,
        "streak": {
            "consecutiveDays": 7,
            "increased": true
        },
        "newAchievements": [
            {
                "id": "ach-001",
                "name": "Racha de 7 días",
                "description": "Completa todas tus tareas por 7 días
consecutivos",
                "xpReward": 100
            }
        ],
        "leveledUp": false
    },
    "nextInstance": {
        "id": 52,
        "name": "Estudiar vocabulario",
        "state": "pending",
        "limitDate": "2025-01-16T00:00:00Z",
        "frequency": 1
    }
}

Response 400:
{
    "error": "TASK_ALREADY_COMPLETED",
    "message": "Esta tarea ya fue completada hoy"
}

Response 404:
{
    "error": "TASK_NOT_FOUND",
```

```
    "message": "La tarea no existe"
}
```

---

13. GET /tasks/daily-summary

Obtener resumen de tareas diarias del día actual.

```
Headers: Authorization: Bearer <token>

Response 200:
{
    "date": "2025-01-15",
    "totalTasks": 8,
    "completedTasks": 5,
    "pendingTasks": 3,
    "completionPercentage": 62.5,
    "tasks": [
        {
            "id": 1,
            "name": "Estudiar vocabulario",
            "state": "completed",
            "difficultyLevel": "easy",
            "goalName": "Aprender inglés"
        },
        {
            "id": 2,
            "name": "Hacer ejercicio",
            "state": "pending",
            "difficultyLevel": "high",
            "goalName": "Vida saludable"
        }
    ]
}
```

---

14. GET /tasks/completions

Historial de completaciones (usado para reportes).

```
Headers: Authorization: Bearer <token>

Query Params:
- from: date (requerido, "2025-01-01")
- to: date (requerido, "2025-01-31")
- taskId: int (opcional)

Response 200:
{
```

```
    "data": [
        {
            "id": 1,
            "taskId": 1,
            "taskName": "Estudiar vocabulario",
            "completedAt": "2025-01-15T18:00:00Z",
            "xpAwarded": 25
        }
    ],
    "summary": {
        "totalCompleted": 45,
        "totalXpEarned": 1125
    }
}
```

MICROSERVICIO 3: GAMIFICATION SERVICE (FastAPI + MongoDB + Beanie/Motor)

Responsabilidades: XP, niveles, rachas, logros, historial de experiencia, reportes semanales, tendencias.

Colecciones MongoDB:

```
# profile_progress
{
    "_id": ObjectId,
    "user_id": "uuid-string",          # referencia al Identity Service
    "xp_total": 1250,
    "current_level": 5,
    "created_at": datetime,
    "updated_at": datetime
}

# streaks
{
    "_id": ObjectId,
    "user_id": "uuid-string",
    "consecutive_days": 7,
    "last_day": "2025-01-15",           # último día que completó todo
    "max_streak": 15,                    # racha máxima histórica
    "updated_at": datetime
}

# xp_history
{
    "_id": ObjectId,
    "user_id": "uuid-string",
    "xp_awarded": 35,
    "source": "task_completion",         # task_completion, achievement,
challenge_win
    "source_id": "task-1",
    "difficulty": "easy",
```

```
    "streak_bonus": 5,
    "created_at": datetime
}

# achievements (catálogo)
{
    "_id": ObjectId,
    "code": "STREAK_7",
    "name": "Racha de 7 días",
    "description": "Completa todas tus tareas por 7 días consecutivos",
    "threshold_value": 7,
    "condition_type": "streak",          # streak, tasks_completed, level
    "xp_reward": 100
}

# user_achievements
{
    "_id": ObjectId,
    "user_id": "uuid-string",
    "achievement_id": ObjectId,
    "achievement_code": "STREAK_7",
    "rewarded_at": datetime
}

# weekly_reports
{
    "_id": ObjectId,
    "user_id": "uuid-string",
    "start_week": "2025-01-13",
    "end_week": "2025-01-19",
    "completed_tasks": 35,
    "total_tasks": 42,
    "completion_percentage": 83.3,
    "xp_earned": 450,
    "level_reached": 5,
    "trend": "increasing",              # increasing, stable, decreasing
    "created_at": datetime
}
```

Endpoints:

---

1. POST /gamification/task-completed

Endpoint principal que llama el Goal Service cuando se completa una tarea. Ejecuta toda la lógica de gamificación.

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body:
```

```
{
    "user_id": "uuid-string",
    "task_id": 1,
    "difficulty": "easy",               // "easy" | "middle" | "high"
    "frequency": 1,                      // frecuencia de la tarea
    "all_daily_tasks_completed": true    // si el Goal Service verificó que se
completaron todas las diarias
}

Lógica interna:
1. Calcular XP base según dificultad:
   - easy = 10, middle = 25, high = 50
2. Obtener racha actual del usuario
3. Si all_daily_tasks_completed == true:
   - Si lastDay == ayer → consecutiveDays += 1
   - Si lastDay == hoy → no cambiar
   - Si lastDay < ayer → consecutiveDays = 1
4. Calcular bonus de racha: streak_bonus = consecutiveDays * 2
5. xp_total = xp_base + streak_bonus
6. Actualizar profile_progress.xp_total
7. Verificar si sube de nivel (fórmula: nivel = floor(xp_total / 100) + 1)
8. Evaluar logros no desbloqueados
9. Registrar en xp_history
10. Retornar resultado

Response 200:
{
    "xp_awarded": 35,
    "xp_breakdown": {
        "base": 10,
        "streak_bonus": 14,
        "achievement_bonus": 100
    },
    "total_xp": 1250,
    "current_level": 5,
    "previous_level": 4,
    "leveled_up": true,
    "streak": {
        "consecutive_days": 7,
        "increased": true,
        "max_streak": 15
    },
    "new_achievements": [
        {
            "id": "ObjectId-string",
            "code": "STREAK_7",
            "name": "Racha de 7 días",
            "description": "Completa todas tus tareas por 7 días consecutivos",
            "xp_reward": 100
        }
    ]
}
```

---

2. POST /gamification/challenge-completed

Endpoint que llama el Challenge Service cuando se determina un ganador.

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body:
{
    "user_id": "uuid-string",
    "challenge_id": 1,
    "result": "win",                // "win" | "lose" | "draw"
    "xp_reward": 200
}

Response 200:
{
    "xp_awarded": 200,
    "total_xp": 1450,
    "current_level": 6,
    "leveled_up": true,
    "new_achievements": []
}
```

---

3. GET /gamification/profile/{user_id}

Obtener perfil de progreso completo del usuario.

```
Headers: Authorization: Bearer <token>

Path Params:
- user_id: string (UUID)

Response 200:
{
    "user_id": "uuid-string",
    "xp_total": 1250,
    "current_level": 5,
    "xp_to_next_level": 50,             // cuánto falta para el próximo nivel
    "xp_progress_percentage": 50.0,      // porcentaje dentro del nivel actual
    "streak": {
        "consecutive_days": 7,
        "max_streak": 15,
        "last_day": "2025-01-15"
    },
    "achievements_count": 12,
    "total_achievements": 30             // total de logros disponibles
}
```

```
Response 404:
{
    "error": "PROFILE_NOT_FOUND",
    "message": "No se encontró perfil de gamificación para este usuario"
}
```

---

4. POST /gamification/profile

Crear perfil de gamificación cuando se registra un usuario nuevo (llamado por Identity Service o Gateway).

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body:
{
    "user_id": "uuid-string"
}

Response 201:
{
    "user_id": "uuid-string",
    "xp_total": 0,
    "current_level": 1,
    "streak": {
        "consecutive_days": 0,
        "max_streak": 0
    }
}
```

---

5. GET /gamification/achievements/{user_id}

Obtener logros del usuario.

```
Headers: Authorization: Bearer <token>

Path Params:
- user_id: string (UUID)

Query Params:
- unlocked: boolean (opcional, true=solo desbloqueados, false=solo bloqueados)

Response 200:
{
    "unlocked": [
        {
            "id": "ObjectId-string",
            "code": "STREAK_7",
```

```
            "name": "Racha de 7 días",
            "description": "Completa todas tus tareas por 7 días consecutivos",
            "xp_reward": 100,
            "rewarded_at": "2025-01-15T18:00:00Z"
        }
    ],
    "locked": [
        {
            "id": "ObjectId-string",
            "code": "STREAK_30",
            "name": "Racha de 30 días",
            "description": "Completa todas tus tareas por 30 días
consecutivos",
            "xp_reward": 500,
            "progress": 7,               // progreso actual hacia el logro
            "threshold": 30
        }
    ]
}
```

---

6. GET /gamification/xp-history/{user_id}

Historial de experiencia obtenida.

```
Headers: Authorization: Bearer <token>

Path Params:
- user_id: string (UUID)

Query Params:
- from: date (opcional)
- to: date (opcional)
- source: string (opcional: "task_completion", "achievement", "challenge_win")
- page: int (default 1)
- limit: int (default 20)

Response 200:
{
    "data": [
        {
            "id": "ObjectId-string",
            "xp_awarded": 35,
            "source": "task_completion",
            "source_id": "task-1",
            "difficulty": "easy",
            "streak_bonus": 14,
            "created_at": "2025-01-15T18:00:00Z"
        }
    ],
    "pagination": {
```

```
        "page": 1,
        "limit": 20,
        "total": 200
    }
}
```

---

7. GET /gamification/streak/{user_id}

Obtener información de racha actual.

```
Headers: Authorization: Bearer <token>

Path Params:
- user_id: string (UUID)

Response 200:
{
    "user_id": "uuid-string",
    "consecutive_days": 7,
    "max_streak": 15,
    "last_day": "2025-01-15",
    "streak_active": true                // si lastDay == hoy o ayer
}
```

---

8. POST /gamification/streak/check-reset

Endpoint para un CRON job diario que reinicia rachas de usuarios que no completaron tareas ayer.

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body:
{
    "user_ids": ["uuid-1", "uuid-2"]     // usuarios que NO completaron tareas
ayer
}

Response 200:
{
    "reset_count": 2,
    "reset_users": ["uuid-1", "uuid-2"]
}
```

---

9. GET /gamification/leaderboard

Ranking de usuarios por XP.

```
Headers: Authorization: Bearer <token>

Query Params:
- limit: int (default 10, max 50)

Response 200:
{
    "leaderboard": [
        {
            "rank": 1,
            "user_id": "uuid-string",
            "xp_total": 5000,
            "current_level": 15,
            "consecutive_days": 30
        },
        {
            "rank": 2,
            "user_id": "uuid-string",
            "xp_total": 4500,
            "current_level": 13,
            "consecutive_days": 12
        }
    ]
}
```

---

10. POST /gamification/reports/weekly

Generar reporte semanal (llamado por CRON job o manualmente).

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body:
{
    "user_id": "uuid-string",
    "start_week": "2025-01-13",
    "end_week": "2025-01-19",
    "completed_tasks": 35,
    "total_tasks": 42
}

Lógica interna:
1. Calcular porcentaje de cumplimiento
2. Obtener XP ganada en la semana desde xp_history
3. Obtener nivel alcanzado
4. Comparar con reporte anterior para calcular tendencia
5. Guardar reporte

Response 201:
```

```
{
    "id": "ObjectId-string",
    "user_id": "uuid-string",
    "start_week": "2025-01-13",
    "end_week": "2025-01-19",
    "completed_tasks": 35,
    "total_tasks": 42,
    "completion_percentage": 83.3,
    "xp_earned": 450,
    "level_reached": 5,
    "trend": "increasing"
}
```

---

11. GET /gamification/reports/weekly/{user_id}

Obtener reportes semanales del usuario.

```
Headers: Authorization: Bearer <token>

Path Params:
- user_id: string (UUID)

Query Params:
- last: int (opcional, últimos N reportes, default 4)

Response 200:
{
    "reports": [
        {
            "id": "ObjectId-string",
            "start_week": "2025-01-13",
            "end_week": "2025-01-19",
            "completed_tasks": 35,
            "total_tasks": 42,
            "completion_percentage": 83.3,
            "xp_earned": 450,
            "level_reached": 5,
            "trend": "increasing"
        },
        {
            "id": "ObjectId-string",
            "start_week": "2025-01-06",
            "end_week": "2025-01-12",
            "completed_tasks": 28,
            "total_tasks": 40,
            "completion_percentage": 70.0,
            "xp_earned": 320,
            "level_reached": 4,
            "trend": "stable"
        }
```

```
    ]
}
```

---

12. GET /gamification/trends/{user_id}

Obtener tendencia de productividad.

```
Headers: Authorization: Bearer <token>

Path Params:
- user_id: string (UUID)

Query Params:
- weeks: int (default 4, número de semanas a analizar)

Response 200:
{
    "user_id": "uuid-string",
    "current_trend": "increasing",
    "trend_data": [
        {
            "week": "2025-01-13",
            "completion_percentage": 83.3,
            "xp_earned": 450
        },
        {
            "week": "2025-01-06",
            "completion_percentage": 70.0,
            "xp_earned": 320
        },
        {
            "week": "2024-12-30",
            "completion_percentage": 65.0,
            "xp_earned": 280
        },
        {
            "week": "2024-12-23",
            "completion_percentage": 55.0,
            "xp_earned": 200
        }
    ],
    "analysis": {
        "avg_completion": 68.3,
        "best_week": "2025-01-13",
        "worst_week": "2024-12-23"
    }
}
```

---

13. GET /gamification/achievements/catalog

Obtener catálogo completo de logros disponibles.

```
Headers: Authorization: Bearer <token>

Response 200:
{
    "achievements": [
        {
            "id": "ObjectId-string",
            "code": "STREAK_7",
            "name": "Racha de 7 días",
            "description": "Completa todas tus tareas por 7 días consecutivos",
            "condition_type": "streak",
            "threshold_value": 7,
            "xp_reward": 100
        },
        {
            "id": "ObjectId-string",
            "code": "TASKS_100",
            "name": "Centenario",
            "description": "Completa 100 tareas",
            "condition_type": "tasks_completed",
            "threshold_value": 100,
            "xp_reward": 250
        }
    ]
}
```

---

14. POST /gamification/achievements (ADMIN)

Crear un nuevo logro en el catálogo.

```
Headers: Authorization: Bearer <token> (rol ADMIN)

Request Body:
{
    "code": "STREAK_30",
    "name": "Racha de 30 días",
    "description": "Completa todas tus tareas por 30 días consecutivos",
    "condition_type": "streak",          // "streak" | "tasks_completed" |
"level"
    "threshold_value": 30,
    "xp_reward": 500
}

Response 201:
{
```

```
    "id": "ObjectId-string",
    "code": "STREAK_30",
    "name": "Racha de 30 días",
    "description": "...",
    "condition_type": "streak",
    "threshold_value": 30,
    "xp_reward": 500
}
```

MICROSERVICIO 4: CHALLENGE SERVICE (NestJS + PostgreSQL + Prisma)

Responsabilidades: CRUD de desafíos, aceptar/rechazar, monitorear progreso, determinar ganador.

Schema Prisma:

```
enum ChallengeState {
    pending
    active
    completed
    rejected
    cancelled
}

enum ChallengeCondition {
    percent      // mayor porcentaje de cumplimiento
    numTasks     // mayor cantidad de tareas completadas
    level        // mayor nivel alcanzado
}

enum ChallengeResult {
    challenger_wins
    opponent_wins
    draw
}

model Challenge {
    id              Int                 @id @default(autoincrement())
    challengerId    String              // UUID del retador
    opponentId      String              // UUID del rival
    condition       ChallengeCondition
    state           ChallengeState      @default(pending)
    durationDays    Int                 // duración en días
    startDate       DateTime?
    endDate         DateTime?
    result          ChallengeResult?
    createdAt       DateTime            @default(now())
    updatedAt       DateTime            @updatedAt
    progress        ChallengeProgress[]
}
```

```
model ChallengeProgress {
    id              Int       @id @default(autoincrement())
    challengeId     Int
    challenge       Challenge @relation(fields: [challengeId], references:
[id])
    userId          String    // UUID
    tasksCompleted  Int       @default(0)
    totalTasks      Int       @default(0)
    xpEarned        Int       @default(0)
    updatedAt       DateTime  @updatedAt
}
```

Endpoints:

---

1. POST /challenges

Crear un desafío contra otro usuario.

```
Headers: Authorization: Bearer <token>

Request Body:
{
    "opponentId": "uuid-string",            // requerido
    "condition": "percent",                  // requerido: "percent" |
"numTasks" | "level"
    "durationDays": 7                        // requerido, duración en días
}

Lógica interna:
1. Verificar que el opponent existe (llamar GET /users/{opponentId} del
Identity Service)
2. Verificar que no haya un desafío activo entre ambos
3. Crear desafío en estado "pending"
4. Retornar

Response 201:
{
    "id": 1,
    "challengerId": "uuid-string",
    "opponentId": "uuid-string",
    "condition": "percent",
    "state": "pending",
    "durationDays": 7,
    "startDate": null,
    "endDate": null,
    "createdAt": "2025-01-15T10:30:00Z"
}

Response 404:
```

```
{
    "error": "OPPONENT_NOT_FOUND",
    "message": "El usuario rival no existe"
}

Response 409:
{
    "error": "ACTIVE_CHALLENGE_EXISTS",
    "message": "Ya existe un desafío activo entre estos usuarios"
}
```

---

2. GET /challenges

Listar desafíos del usuario autenticado.

```
Headers: Authorization: Bearer <token>

Query Params:
- state: string (opcional: "pending", "active", "completed", "rejected")
- role: string (opcional: "challenger", "opponent", "all")
- page: int (default 1)
- limit: int (default 10)

Response 200:
{
    "data": [
        {
            "id": 1,
            "challengerId": "uuid-string",
            "challengerName": "Juan",            // obtenido del Identity
Service
            "opponentId": "uuid-string",
            "opponentName": "María",
            "condition": "percent",
            "state": "active",
            "durationDays": 7,
            "startDate": "2025-01-15T00:00:00Z",
            "endDate": "2025-01-22T00:00:00Z",
            "myRole": "challenger",
            "createdAt": "2025-01-15T10:30:00Z"
        }
    ],
    "pagination": {
        "page": 1,
        "limit": 10,
        "total": 3
    }
}
```

---

3. GET /challenges/:id

Obtener detalle de un desafío con progreso.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "id": 1,
    "challengerId": "uuid-string",
    "challengerName": "Juan",
    "opponentId": "uuid-string",
    "opponentName": "María",
    "condition": "percent",
    "state": "active",
    "durationDays": 7,
    "startDate": "2025-01-15T00:00:00Z",
    "endDate": "2025-01-22T00:00:00Z",
    "daysRemaining": 4,
    "result": null,
    "progress": {
        "challenger": {
            "userId": "uuid-string",
            "name": "Juan",
            "tasksCompleted": 15,
            "totalTasks": 21,
            "completionPercentage": 71.4,
            "xpEarned": 250
        },
        "opponent": {
            "userId": "uuid-string",
            "name": "María",
            "tasksCompleted": 18,
            "totalTasks": 21,
            "completionPercentage": 85.7,
            "xpEarned": 310
        }
    },
    "createdAt": "2025-01-15T10:30:00Z"
}

Response 404:
{
    "error": "CHALLENGE_NOT_FOUND",
    "message": "El desafío no existe"
}
```

4. PATCH /challenges/:id/accept

Aceptar un desafío pendiente.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Lógica:
1. Verificar que el usuario autenticado es el opponent
2. Cambiar state → active
3. Establecer startDate = ahora, endDate = ahora + durationDays
4. Crear registros ChallengeProgress para ambos usuarios

Response 200:
{
    "id": 1,
    "state": "active",
    "startDate": "2025-01-15T18:00:00Z",
    "endDate": "2025-01-22T18:00:00Z",
    "message": "Desafío aceptado exitosamente"
}

Response 403:
{
    "error": "NOT_OPPONENT",
    "message": "Solo el rival puede aceptar el desafío"
}

Response 400:
{
    "error": "INVALID_STATE",
    "message": "El desafío no está en estado pendiente"
}
```

---

5. PATCH /challenges/:id/reject

Rechazar un desafío pendiente.

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "id": 1,
    "state": "rejected",
```

```
    "message": "Desafío rechazado"
}
```

---

6. PATCH /challenges/:id/cancel

Cancelar un desafío (solo el challenger si aún está pending).

```
Headers: Authorization: Bearer <token>

Path Params:
- id: int

Response 200:
{
    "id": 1,
    "state": "cancelled",
    "message": "Desafío cancelado"
}
```

---

7. POST /challenges/update-progress

Endpoint interno llamado cuando se completa una tarea para actualizar el progreso de los desafíos activos del usuario.

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body:
{
    "user_id": "uuid-string",
    "tasks_completed_today": 5,
    "total_tasks_today": 8,
    "xp_earned": 35
}

Lógica:
1. Buscar todos los desafíos activos donde este usuario participe
2. Actualizar tasksCompleted y totalTasks en ChallengeProgress
3. Si algún desafío alcanzó endDate → calcular ganador

Response 200:
{
    "updated_challenges": [1, 3],
    "completed_challenges": [
        {
            "id": 3,
            "result": "challenger_wins",
            "xp_reward_winner": 200,
```

```
            "xp_reward_loser": 50
        }
    ]
}
```

---

8. POST /challenges/check-expired

CRON endpoint para verificar desafíos que han expirado y calcular ganador.

```
Headers: X-Internal-Service-Key: <shared-secret>

Request Body: (vacío)

Lógica:
1. Buscar todos los desafíos con state=active y endDate <= now()
2. Para cada uno, calcular ganador según condition
3. Llamar al Gamification Service para otorgar XP
4. Actualizar state → completed

Response 200:
{
    "processed": 2,
    "results": [
        {
            "challengeId": 1,
            "result": "opponent_wins",
            "winnerId": "uuid-string",
            "loserId": "uuid-string"
        }
    ]
}
```

MICROSERVICIO 5: API GATEWAY (Spring Cloud Gateway)

Responsabilidades: Enrutamiento, validación JWT centralizada, rate limiting.

Tabla de enrutamiento:

Ruta Externa	Microservicio Destino	Autenticación
`/api/auth/**`	Identity Service	NO (público)
`/api/users/**`	Identity Service	SÍ
`/api/audit/**`	Identity Service	SÍ (ADMIN)
`/api/goals/**`	Goal & Task Service	SÍ

Ruta Externa	Microservicio Destino	Autenticación
`/api/tasks/**`	Goal & Task Service	SÍ
`/api/challenges/**`	Challenge Service	SÍ
`/api/gamification/**`	Gamification Service	SÍ

DOCKER COMPOSE COMPLETO

```yaml
version: '3.8'

services:
  # ============ BASES DE DATOS ============
  mysql-identity:
    image: mysql:8.0
    container_name: mysql-identity
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: identity_db
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - app-network

  postgres-tasks:
    image: postgres:15
    container_name: postgres-tasks
    environment:
      POSTGRES_DB: tasks_db
      POSTGRES_USER: tasks_user
      POSTGRES_PASSWORD: tasks123
    ports:
      - "5432:5432"
    volumes:
      - postgres_tasks_data:/var/lib/postgresql/data
    networks:
      - app-network

  postgres-challenges:
    image: postgres:15
    container_name: postgres-challenges
    environment:
      POSTGRES_DB: challenges_db
      POSTGRES_USER: challenges_user
      POSTGRES_PASSWORD: challenges123
    ports:
      - "5433:5432"
    volumes:
```

```yaml
      - postgres_challenges_data:/var/lib/postgresql/data
    networks:
      - app-network

  mongo-gamification:
    image: mongo:7
    container_name: mongo-gamification
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    networks:
      - app-network

  # ============ MICROSERVICIOS ============
  api-gateway:
    build: ./api-gateway
    container_name: api-gateway
    ports:
      - "8080:8080"
    environment:
      IDENTITY_SERVICE_URL: http://identity-service:8081
      TASK_SERVICE_URL: http://task-service:3001
      GAMIFICATION_SERVICE_URL: http://gamification-service:8000
      CHALLENGE_SERVICE_URL: http://challenge-service:3002
      JWT_SECRET: your-shared-jwt-secret-key
    depends_on:
      - identity-service
      - task-service
      - gamification-service
      - challenge-service
    networks:
      - app-network

  identity-service:
    build: ./identity-service
    container_name: identity-service
    ports:
      - "8081:8081"
    environment:
      DB_HOST: mysql-identity
      DB_PORT: 3306
      DB_NAME: identity_db
      DB_USER: root
      DB_PASSWORD: root123
      JWT_SECRET: your-shared-jwt-secret-key
      JWT_EXPIRATION: 3600
      GAMIFICATION_SERVICE_URL: http://gamification-service:8000
    depends_on:
      - mysql-identity
    networks:
      - app-network
```

```yaml
  task-service:
    build: ./task-service
    container_name: task-service
    ports:
      - "3001:3001"
    environment:
      DATABASE_URL: postgresql://tasks_user:tasks123@postgres-
tasks:5432/tasks_db
      GAMIFICATION_SERVICE_URL: http://gamification-service:8000
      CHALLENGE_SERVICE_URL: http://challenge-service:3002
      IDENTITY_SERVICE_URL: http://identity-service:8081
      INTERNAL_SERVICE_KEY: internal-secret-key
    depends_on:
      - postgres-tasks
    networks:
      - app-network

  gamification-service:
    build: ./gamification-service
    container_name: gamification-service
    ports:
      - "8000:8000"
    environment:
      MONGO_URL: mongodb://mongo-gamification:27017/gamification_db
      INTERNAL_SERVICE_KEY: internal-secret-key
    depends_on:
      - mongo-gamification
    networks:
      - app-network

  challenge-service:
    build: ./challenge-service
    container_name: challenge-service
    ports:
      - "3002:3002"
    environment:
      DATABASE_URL: postgresql://challenges_user:challenges123@postgres-
challenges:5432/challenges_db
      GAMIFICATION_SERVICE_URL: http://gamification-service:8000
      IDENTITY_SERVICE_URL: http://identity-service:8081
      INTERNAL_SERVICE_KEY: internal-secret-key
    depends_on:
      - postgres-challenges
    networks:
      - app-network

volumes:
  mysql_data:
  postgres_tasks_data:
  postgres_challenges_data:
  mongo_data:

networks:
```

```
    app-network:
      driver: bridge
```

Reparto por desarrollador:

Desarrollador	Servicios	Endpoints
Dev A (Java)	API Gateway + Identity Service	10 endpoints + routing
Dev B (NestJS)	Goal/Task Service + Challenge Service	14 + 8 = 22 endpoints
Dev C (Python)	Gamification Service	14 endpoints